

Q1. Mark the following statements as true or false.

1. Anonymous routes are created directly with Route objects and do not require a registered string name. ()
2. Named routes can improve organization when route names and their argument contracts are kept clear. ()
3. The Navigator widget in Flutter manages a stack of Route objects. ()
4. pushReplacement installs a new route and removes the current route from Back history. ()
5. Named routes can be used for easier navigation management. ()
6. Navigator.pop() requires a mandatory data argument on every call. ()
7. Flutter can support deep linking, which opens a specific screen from a URL or path. ()
8. MaterialPageRoute creates a platform-appropriate full-page route transition. ()
9. pushAndRemoveUntil() can push a new route and remove earlier routes according to a predicate. ()
10. Named routes can be registered through the MaterialApp routes property. ()
11. Navigator.push() returns a Future that completes when the pushed route is popped. ()
12. onGenerateRoute can handle dynamic route creation and an unknown-route fallback. ()
13. A route is a screen represented by a widget and displayed by a Navigator. ()
14. push adds a route to the stack, while pop removes the top route from the relevant Navigator. ()
15. Navigator.pushNamed() receives a BuildContext and route name, with arguments supplied optionally. ()
16. Navigator.pushNamed() can navigate to a registered named route. ()
17. Navigator.pop() is used only to close the entire application. ()
18. MaterialApp can configure the app Navigator and its route entry points. ()
19. Navigator.pushReplacement() replaces the current screen in the route history. ()
20. Navigator.pushAndRemoveUntil() can remove multiple routes from the stack. ()
21. The BuildContext passed to Navigator.of(context) is optional in most cases. ()
22. In Flutter navigation, a screen is represented by a Route. ()
23. Navigator.canPop() returns true when a previous route can be revealed. ()



24. Hero animations can animate a matching widget between two routes during navigation. ()
25. Hero tags should be unique enough within each participating route to identify the intended shared element. ()
26. PageRouteBuilder can provide a custom transition for an entire page route. ()
27. An image used by a Hero still has to be available through the app's asset or network configuration. ()
28. A TabBar and TabBarView should have matching lengths when they are controlled by the same tab controller. ()
29. An IndexedStack can keep the state of inactive pages while changing which child is visible. ()
30. A NavigationBar shell normally stores the selected destination index in State so a selection rebuilds the body. ()
31. A TextEditingController owned by a State object should be disposed when that State is disposed. ()
32. AlertDialog can return a nullable Boolean so the caller can distinguish confirm from cancel or dismissal. ()
33. SimpleDialog can return a nullable selection value when the user chooses an option or dismisses the dialog. ()
34. A typed modal bottom sheet can return an enum value through its Future. ()
35. The caller should handle null after awaiting a dialog or bottom sheet because dismissal is a valid outcome. ()
36. showDatePicker returns a nullable DateTime because the user can cancel the picker. ()
37. showTimePicker returns a nullable TimeOfDay because cancellation does not create a selected time. ()
38. A date picker should receive a legal firstDate, lastDate, and initialDate relationship. ()
39. The current XFile API represents a selected local image; it does not by itself represent a completed upload. ()
40. When image selection is cancelled, a null XFile can be used to leave the previous preview unchanged. ()
41. Android activity recreation can cause a picker result to need explicit lost-data recovery. ()
42. A NavigatorObserver can record push, pop, replace, and remove events without rendering the screens itself. ()
43. Route names make observer logs and widget-test assertions easier to interpret. ()
44. A widget test can use pumpAndSettle after navigation and asynchronous work before checking the final screen. ()



45. BuildContext identifies a location in the widget tree; it is not a suitable data model for a Course. ()
46. Navigator.pop(context) removes the route at the top of the relevant navigator stack. ()
47. pushAndRemoveUntil can preserve a newly opened home route while removing all routes below it when the predicate is false. ()
48. popUntil and pushReplacement express different history changes and should not be selected only because they both change the visible screen. ()
49. Named and anonymous routes can coexist, but the application should keep their contracts understandable. ()
50. A caller awaiting a typed result should handle both a returned value and a null cancellation path. ()
51. Using a BuildContext after an await without checking its lifecycle can lead to invalid navigation or feedback calls. ()
52. A Hero transition changes the animation between routes; it does not merge the two routes into one route. ()
53. A modal dialog or bottom sheet is presented above the current content and normally completes with a nullable result. ()
54. maybePop is not guaranteed to remove a route, so code must not assume that its call always changes the stack ()
55. Preserving tab state requires an ownership strategy such as IndexedStack or a suitable state-preserving navigation shell. ()
56. A picker or date dialog should preserve the previous value when the user cancels rather than replacing it with an invented value. ()
57. The smallest correct navigation API is the one whose history semantics match the required Back behavior. ()
58. A test that checks only colors cannot prove that a typed route result returned to the correct caller. ()
59. A strong navigation explanation names the action, the route-stack effect, the result type, and the lifecycle guard when one is needed. ()



60. A navigation diagram is incomplete if it shows only the final screen and omits the route-stack or result changes that produced it. ()



Q2. Choose the correct answer from the following multiple choices

1. Which concepts allow a mobile user to move between different pages?
 - A. Navigation
 - B. Routing
 - C. A and B
 - D. None of the mentioned
2. In Flutter, screens and pages are known as ____, and they are widgets displayed by a Navigator.
 - A. Navigators
 - B. routes
 - C. A and B
 - D. None of the mentioned
3. Flutter provides MaterialPageRoute and which two methods to demonstrate navigation between routes?
 - A. Navigator.push()
 - B. Navigator.pop()
 - C. A and B
 - D. None of the mentioned
4. Navigate to one route from another route by using the ____ method.
 - A. Navigator.push()
 - B. Navigator.pop()
 - C. Push()
 - D. Pop()
5. Which method removes the current route from the Navigator stack?
 - A. pop()
 - B. push()
 - C. remove()
 - D. None of the mentioned
6. Which line opens SecondRoute_Roq and adds it to the stack without sending data?
 - A. Navigator.pop(context, MaterialPageRoute(SecondRoute_Roq()))
 - B. Navigator.push(context, MaterialPageRoute(builder: (context) => SecondRoute_Roq()))
 - C. Navigator.push(context, MaterialPageRoute(SecondRoute_Roq(), builder: context))
 - D. Navigator.pushNamed(context, MaterialPageRoute(SecondRoute_Roq()))



7. What is the purpose of the Navigator widget in Flutter?

- A. Provides push and pop methods
- B. Manages a stack of routes
- C. Navigates between screens
- D. All of the mentioned

8. Flutter supports which two broad route styles?

- A. Named routes
- B. Anonymous routes
- C. Named and anonymous routes
- D. None of the mentioned

9. To read arguments passed to a named route, which API is used?

- A. ModalRoute.of(context)
- B. ModalRoute()
- C. Modal
- D. None of the mentioned

10. Which object is commonly used to handle navigation in a Flutter app?

- A. Navigator
- B. NavigationView
- C. RouteManager
- D. NavController

11. Which widget is commonly used to define a material route in Flutter?

- A. RoutePage
- B. MaterialPageRoute
- C. RouteNavigator
- D. PageViewRoute

12. Which MaterialApp property registers named routes?

- A. routeList
- B. namedRoutes
- C. routes
- D. onRouteChange

13. Which method can remove previous routes and push a new one?

- A. Navigator.pushNamed()
- B. Navigator.popUntil()



C. Navigator.pushAndRemoveUntil()

D. Navigator.replaceAll()

14. What does Navigator.push() return when the new screen is popped?

A. A boolean value

B. An integer value

C. A Future with the result

D. Nothing

15. Which statement about onGenerateRoute is correct?

A. It automatically generates UI widgets

B. It handles dynamic route generation

C. It is only for web applications

D. It disables named routes

16. At the root route, which guarded operation can leave the route in place when no pop is allowed?

A. Navigator.maybePop()

B. Navigator.push()

C. Navigator.popUntil() with a new route

D. Navigator.replaceAll()

17. Which method is used to navigate to a named route?

A. Navigator.loadNamed()

B. Navigator.pushNamed()

C. Navigator.routeTo()

D. Navigator.startNamed()

18. A sort bottom sheet offers enum values and may be dismissed. What is a clear contract?

A. Future<SortType?>

B. Future<void> that hides all choices

C. Future<Widget> with a database row

D. Future<BuildContext?>

19. What is the safest handling after awaiting a picker or dialog in a State object?

A. Check mounted before using context or calling setState

B. Assume the State is still mounted

C. Call dispose manually

D. Push the same route again



20. A date picker accepts an initial date outside its legal range. What should be fixed?

- A. Choose an initial date within firstDate and lastDate
- B. Replace DateTime with BuildContext
- C. Use pushReplacement
- D. Ignore the range because the dialog repairs it

21. The user cancels time selection. What should the screen display?

- A. The previously stored time
- B. A new time chosen at random
- C. A popped route name
- D. The database password

22. A modern image picker returns XFile?. What does null most directly indicate?

- A. The user did not select an image
- B. The image has already been uploaded
- C. The route stack is empty
- D. The image is a database row

23. An XFile is selected and a preview is shown. What evidence is still needed to claim server persistence?

- A. An accepted upload response and a later server-backed read
- B. The picker returned normally
- C. The preview widget built
- D. The file has a local path

24. An Android picker flow can lose its result after activity recreation. Which additional handling is relevant?

- A. Recover lost picker data when the plugin provides it
- B. Use Navigator.popUntil for every result
- C. Store a BuildContext in the file
- D. Disable all async code

25. What can a NavigatorObserver prove in a widget test?

- A. That route lifecycle events occurred in the expected order
- B. That a server database contains a row
- C. That an image MIME type is valid
- D. That every widget has identical colors

26. After tapping a route and waiting for animations and Futures, which test step is commonly needed before finding the final widget?

- A. pumpAndSettle



- B. jsonEncode
- C. dispose
- D. rootBundle.loadString

27. Which value is most appropriate to pass through a destination constructor?

- A. A typed immutable Course model
- B. A BuildContext belonging to the caller
- C. A mutable Navigator object as domain data
- D. A raw widget tree snapshot

28. What is the main difference between pushReplacement and push?

- A. Replacement removes the current route from the return history
- B. Push always clears every route
- C. Replacement can only open dialogs
- D. Push never changes the stack

29. What is the main difference between popUntil and pushAndRemoveUntil?

- A. The first removes existing top routes; the second pushes a route and then removes routes below according to a predicate
- B. They are identical in every case
- C. Both only animate Hero widgets
- D. Neither changes route history

30. A caller needs to know whether a destination selected Ocean. Which design is strongest?

- A. A typed nullable result returned through Navigator.pop
- B. A global string read from the AppBar
- C. A new route name containing Ocean
- D. A database query from the widget

31. A route receives arguments = 42 but expects a Course object. What should the destination do?

- A. Reject or route to a controlled error because the contract is wrong
- B. Cast 42 to Course without checking
- C. Use 42 as BuildContext
- D. Silently show an unrelated screen

32. Which condition indicates that the current route is not the root route?

- A. Navigator.canPop(context) is true
- B. Navigator.canPop(context) is false
- C. The AppBar has a title



D. A Hero tag is present

33. Why can maybePop be preferable to an unconditional pop in a back action?

- A. It allows the navigator or a route guard to decide whether popping is allowed
- B. It guarantees two routes are removed
- C. It bypasses all lifecycle checks
- D. It returns a database row

34. A dirty edit screen shows a Stay/Discard dialog. What should Stay do?

- A. Close the dialog and keep the edit route
- B. Pop the edit route twice
- C. Delete the draft silently
- D. Replace the app root

35. What should Discard do after the user confirms?

- A. Allow exactly one route pop and remove the unsaved screen
- B. Keep the route and pretend it was saved
- C. Open the same dialog in a loop
- D. Push a duplicate edit route

36. Which widget is the natural destination for a Material 3 three-destination shell?

- A. NavigationBar
- B. TabBarView only
- C. SimpleDialog
- D. ImagePicker

37. Which statement about IndexedStack is correct?

- A. Inactive children can remain mounted while only one is visible
- B. It always disposes inactive children immediately
- C. It can display only route names
- D. It automatically validates API responses

38. A widget has a controller created in initState and used by a visible Search page. Where should cleanup occur?

- A. dispose
- B. build
- C. onGenerateRoute
- D. NavigatorObserver.didPush



39. A modal bottom sheet is dismissed by tapping outside. What result should the caller expect if its generic type is `SortType`?
- A. A nullable `SortType` result
 - B. A guaranteed `SortType`
 - C. A route stack list
 - D. A `FutureBuilder` snapshot only
40. Why is checking `mounted` after awaiting `Navigator.push` important?
- A. The caller may have been removed while the destination was open
 - B. It makes the result non-null
 - C. It registers a Hero asset
 - D. It clears the database
41. Which test most directly checks the complete result flow `Home` → `Details` → choose `Ocean` → `Home` feedback?
- A. Open the route, choose `Ocean`, `pumpAndSettle`, and assert the `Home` feedback
 - B. Assert only that `Home` exists before tapping
 - C. Inspect only the `AppBar` color
 - D. Call `dispose` before navigation
42. A route has a name but receives a missing argument. What is a robust UI outcome?
- A. A controlled unknown/invalid-route fallback
 - B. A blind cast and crash
 - C. A random details screen
 - D. A successful database write
43. Which statement about Hero tags is most accurate?
- A. The source and destination need matching tags for the intended shared element
 - B. Tags are optional text labels that never affect animation
 - C. Tags replace route settings
 - D. Tags determine database ids
44. A date range picker returns a nullable `DateTimeRange`. How should cancellation be treated?
- A. Keep the previous range when the result is null
 - B. Convert null to today automatically
 - C. Pop the entire app
 - D. Send `DELETE` to the server
45. What does a named route provide that a constructor route does not automatically provide?
- A. A string-based entry point that can be centralized or deep-linked



- B. Automatic type safety for all arguments
- C. Automatic persistent storage
- D. Automatic image upload

46. A destination needs an immutable course title and summary. Which constructor design is best?

- A. Required final fields with a typed Course value
- B. An optional BuildContext field
- C. A dynamic map with no validation
- D. A global mutable string

47. A navigation test checks that Login is absent after entering Home. Which evidence is relevant?

- A. The route stack or Back behavior after the transition
- B. Only the Home background color
- C. The number of Hero tags
- D. The font used by Login

48. A user taps a picker button twice while the first dialog is open. Which design concern is most relevant?

- A. Avoiding duplicate modal actions or handling the resulting state deliberately
- B. Changing a SQL column type
- C. Adding a second MySQL connection
- D. Replacing JSON with XML

49. Which approach keeps a search query when the user changes tabs?

- A. Keep the search page state alive in a stable navigation shell
- B. Recreate the page on every selection
- C. Pop the search route on every tap
- D. Store BuildContext in the query

50. A route result is nullable because dismissal is legal. Which condition shows a selected value?

- A. result != null
- B. result == null only
- C. Navigator.canPop(result)
- D. mounted == result

51. What should a navigation API choice be judged against?

- A. The required visible destination and Back/return semantics
- B. The shortest method name only
- C. The color palette
- D. The database engine



52. Which statement best separates navigation data from persistent data?

- A. A route result communicates between active screens; persistence survives a later restart
- B. They are always identical
- C. A route result always writes SQLite
- D. Persistence exists only in BuildContext

53. A widget uses a context after an await but the route was popped. What risk exists?

- A. An invalid context operation or lifecycle error
- B. Automatic route restoration
- C. Automatic type conversion
- D. A guaranteed successful pop

54. What does a successful pop with a value do to the caller?

- A. Completes the caller's push Future with that value
- B. Changes the database schema
- C. Creates a Hero tag
- D. Removes every route below the caller

55. Which statement about modal routes is correct?

- A. They temporarily overlay the current route and can return a value
- B. They always replace the app root
- C. They cannot be dismissed
- D. They are database transactions

56. A screen must keep its previous data if the user cancels a picker. What implementation rule follows?

- A. Assign the new value only when the nullable result is non-null
- B. Assign null on every cancellation
- C. Call pushReplacement on cancellation
- D. Rebuild the whole app without State

57. A strong answer to a navigation tracing task should identify what?

- A. The stack after each action and the data/result path
- B. Only widget colors
- C. Only the final route name
- D. Only the number of buttons

58. A screen opens a modal choice and then updates only when a non-null value returns. What does this design preserve?

- A. The previous state on cancellation
- B. A duplicate route on every dismissal



- C. A database row for every choice
- D. A permanent result after uninstall

59. Which navigation trace is most useful when diagnosing an unexpected Back destination?

- A. Record the stack before and after each push, pop, replacement, or removal
- B. Record only the final widget color
- C. Record only the dialog title
- D. Record the number of Text widgets

60. A details route needs to return a saved item id to its caller. Which contract is clearest?

- A. Navigator.push<int> with Navigator.pop(context, id)
- B. Navigator.push<void> and a global mutable id
- C. A route name that contains the id with no parsing
- D. A BuildContext stored in the item



Q3. Predict the Navigation Code Output

Read the self-contained code first. Then select the exact stack effect, returned value, or visible state.

Code fragment:

```
// Start: [Home]
Navigator.push(context, MaterialPageRoute(builder: (_) => const Courses()));
```

1. After this fragment runs, which route is on top of the stack?

- A. [Home, Courses]
- B. [Courses]
- C. [Home]
- D. []

Code fragment:

```
// Start: [Home, Details]
Navigator.pop(context);
```

2. Which screen becomes visible and what remains in history?

- A. Details remains visible
- B. Home is visible and Details remains below it
- C. Both routes are removed
- D. Home is removed

Code fragment:

```
// Start: [Login]
Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => const Home()));
```

3. Which Back destination remains?

- A. Back returns to Login
- B. Home is visible and Login is not below it
- C. Both routes are removed
- D. Login stays on top

Code fragment:

```
return PopScope<void>(canPop: !hasChanges, onPopInvokedWithResult: (didPop, result) { if (!didPop && hasChanges) showDiscardDialog(); }, child: const EditScreen());
```

4. A system-back attempt occurs while hasChanges is true. Which outcome is expected?

- A. The dirty route is popped immediately
- B. The guard can show its decision UI and keep the route



- C. The Future becomes a database row
- D. Two routes are removed

Code fragment:

```
final theme = await Navigator.push<String>(context, route); // destination: Navigator.pop(context, 'Ocean');
```

5. Which value does the caller receive?

- A. theme is 'Ocean'
- B. theme is false
- C. theme is necessarily null
- D. The route stack resets

Code fragment:

```
final language = await showDialog<String>(context: context, builder: (_) => const SimpleDialog()); // user dismisses
```

6. What result should the caller handle?

- A. language is English
- B. language is null
- C. language is an empty String
- D. An exception is required

Code fragment:

```
IndexedStack(index: index, children: pages) // NavigationBar changes index; Search is one child
```

7. Which page is visible at the selected index?

- A. Search is visible at the selected index
- B. Every page is disposed
- C. A new route is pushed
- D. The route result is null

Code fragment:

```
Hero(tag: 'course-image', child: Image.asset('assets/course.png')) // destination uses the same Hero tag
```

8. Which element participates in the transition?

- A. The matching shared element
- B. The two routes become one route
- C. Only the AppBar
- D. No transition can occur



Code fragment:

```
final value = await showTimePicker(context: context, initialTime: const TimeOfDay(hour: 9, minute: 30));  
if (value == null || !mounted) return;  
setState(() => selectedTime = value);
```

9. A time picker is cancelled while a previous time exists. Which time is rendered?

- A. Cancel sets selectedTime to midnight
- B. Cancel throws
- C. Cancel removes the current route
- D. Cancel leaves selectedTime unchanged

Code fragment:

```
await tester.tap(find.text('Flutter Navigation'));  
await tester.pumpAndSettle();  
await tester.tap(find.text('Ocean'));  
await tester.pumpAndSettle();
```

10. Which final screen and feedback can be visible after the typed result returns?

- A. The test must end on Details
- B. Home can display Selected theme: Ocean
- C. Ocean cannot cross a route boundary
- D. The route result is always null



Q4. Match Navigation Code with the Correct Screen or Result

Match each numbered code/state fragment with the correct labeled visual. Write the letters only; justify the match from the stated state.

Code 1:

```
Navigator.push<void>(context, MaterialPageRoute(builder: (_) => const ProductDetailsScreen()));
```

Code 2:

```
final theme = await Navigator.push<String>(context, route); // destination pops 'Ocean'
```

Code 3:

```
Navigator.pushNamed(context, '/course', arguments: 42); // route validates a String
```

Code 4:

```
IndexedStack(index: 1, children: pages); // Search is a stable child
```

A. Details route: [Home, Products, Details]; Back reveals Products.	B. Home feedback shows: Selected theme: Ocean.
C. Controlled invalid-argument fallback is displayed.	D. Search remains mounted when the selected index changes.

Code 1: ____ Code 2: ____ Code 3: ____ Code 4: ____



Q5. Match Modal or Picker Code with the Correct Surface

Match each numbered code/state fragment with the correct labeled visual. Write the letters only; justify the match from the stated state.

Code 1:

```
showDialog<bool>(context: context, builder: (_) => AlertDialog(...));
```

Code 2:

```
showDialog<String>(context: context, builder: (_) => SimpleDialog(...));
```

Code 3:

```
showModalBottomSheet<SortType>(context: context, builder: (_) => ...);
```

Code 4:

```
showTimePicker(context: context, initialTime: const TimeOfDay(hour: 9, minute: 30));
```

A. AlertDialog: Delete account? with Delete/Cancel/dismiss paths.	B. SimpleDialog: Choose language with a nullable String result.
C. Modal bottom sheet: Sort options with a nullable SortType result.	D. Framework time picker: Select a time; cancellation is nullable.

Code 1: ____ Code 2: ____ Code 3: ____ Code 4: ____



Q6. Correct the Navigation Code to Produce the Required Behavior

1. Login must not reappear through Back after entry.

- Faulty: `Navigator.push(context, MaterialPageRoute(builder: (_) => const Home()));`
- Replace it with `pushReplacement` or a reset flow and state the resulting stack.

Code response:

2. The caller must handle a nullable typed theme result safely.

- Faulty: `final theme = Navigator.push<String>(context, route); showFeedback(theme);`
- Await the push, check null, and check mounted before using context.

Code response:

3. The tab controller, TabBar, and TabBarView must have equal lengths.

- Faulty: `TabController(length: 3)` with two `TabBar` tabs and two `TabBarView` children.
- Correct the counts and show the aligned structure.

Code response:

4. The date/time picker must preserve the previous value on cancellation.

- Faulty: `setState(() => selectedDate = await showDatePicker(...));`
- Store the nullable result and assign only when it is non-null and in range.

Code response:

5. Use the current `image_picker` API and separate local selection from upload.

- Faulty: `PickedFile? selected = await picker.getImage(source: source); await uploadImage(selected!);`
- Use `XFile?`, `pickImage`, cancellation handling, and an explicit upload-success step.

Code response:

6. The widget test must settle asynchronous navigation before asserting Home feedback.

- Faulty: `await tester.tap(find.text('Ocean')); expect(find.text('Selected theme: Ocean'), findsOneWidget);`
- Add the appropriate `pump/pumpAndSettle` sequence and keep the assertion after the result returns.

Code response:



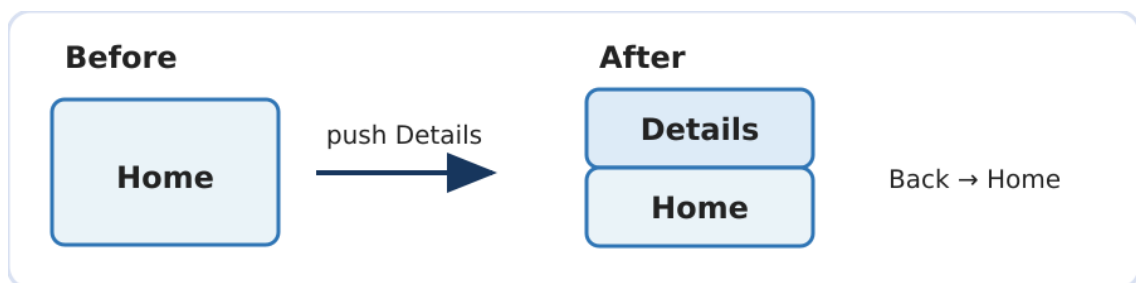
Q7. Navigation Code and Screen State Analysis

State/code description:

```
Navigator.push<void>(context, MaterialPageRoute(builder: (_) => const ProductDetailsScreen()));
```

1. The stack begins [Home]. Which labeled stack state is correct?

A. [Home, Details] — Details is on top	B. [Details] — Home was removed
C. [Home] — no push occurred	D. [Details, Home] — Details is below



Choice: ____ Justification: _____

State/code description:

```
final theme = await Navigator.push<String>(context, route); // destination: Navigator.pop(context, 'Ocean');
```

2. Details returns Ocean. Which labeled Home state includes the feedback?

A. Home — Selected theme: Ocean	B. ThemePicker remains open
C. Home — no feedback can appear	D. Login — result resets the app



Choice: ____ Justification: _____

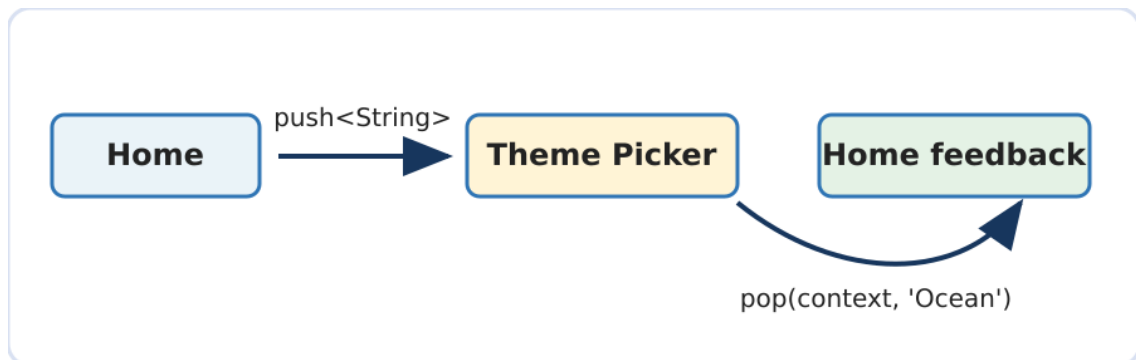


State/code description:

IndexedStack(index: 1, children: pages) with Search as a stable child.

3. Which labeled state keeps the Search query?

A. Search TextField still contains routing	B. Search TextField is recreated empty
C. Search is a modal dialog	D. Login replaces the shell



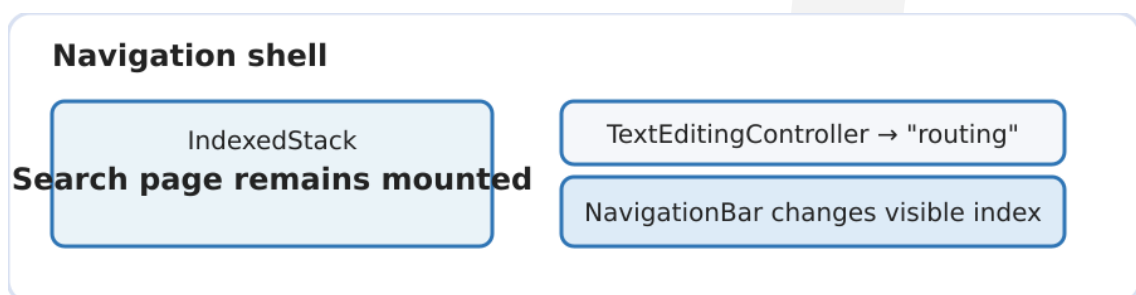
Choice: _____ Justification: _____

State/code description:

```
showDialog<bool>(context: context, builder: (_) => AlertDialog(title: Text('Delete account?')));
```

4. Which labeled modal surface is produced?

A. AlertDialog: Delete account?	B. SimpleDialog: Choose language
C. Bottom sheet: Sort list	D. Time picker: clock surface



Choice: _____ Justification: _____

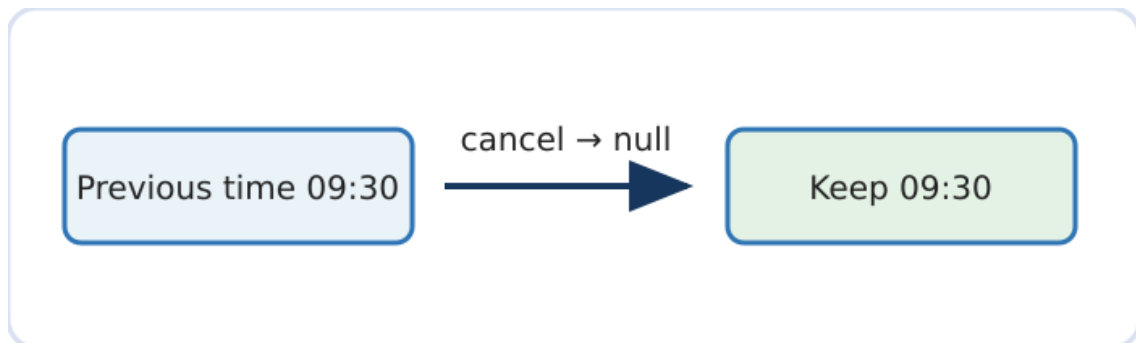


State/code description:

```
showModalBottomSheet<SortType>(context: context, builder: (_) => const SortOptions());
```

5. Which labeled surface is produced?

A. AlertDialog	B. SimpleDialog
C. Bottom sheet: Sort options	D. Time picker



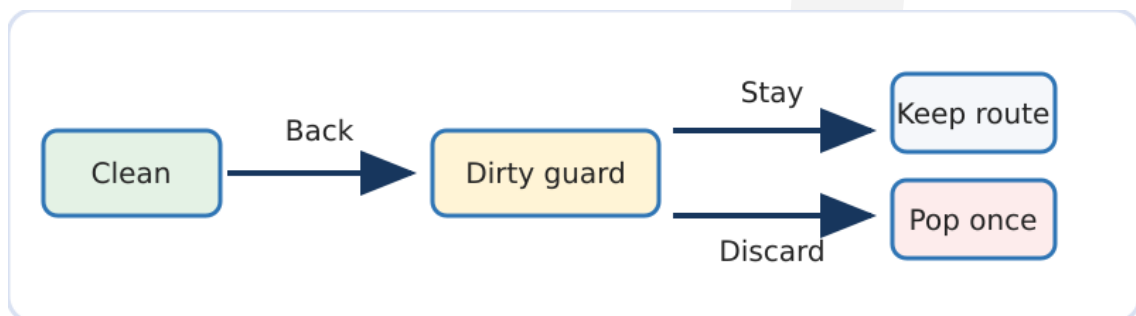
Choice: _____ Justification: _____

State/code description:

```
final selected = await picker.pickImage(source: ImageSource.gallery); if (selected != null && mounted)
setState(() => image = selected);
```

6. An XFile was selected. Which visual state is correct?

A. Profile preview shows the new selected image	B. Previous preview remains despite the real selection
C. Search has no image area	D. The picker replaces the app root



Choice: _____ Justification: _____



State/code description:

```
showTimePicker(context: context, initialTime: const TimeOfDay(hour: 9, minute: 30));
```

7. Which framework surface is waiting for a choice?

A. Clock surface: Select a time	B. Calendar surface: Select a date
C. AlertDialog: Confirmation	D. Bottom sheet: Sort options

Choice: _____ Justification: _____

State/code description:

```
showDatePicker(context: context, initialDate: DateTime(2026, 7, 24), firstDate: DateTime(2025), lastDate: DateTime(2027, 12, 31));
```

8. Which framework surface is waiting for a choice?

A. Calendar surface: Select a date	B. Clock surface: Select a time
C. SimpleDialog: Choose language	D. AlertDialog: Confirmation

Choice: _____ Justification: _____



Q8. Write Flutter Code for the Stated Navigation Contract**1. Products → Details → Back with typed route construction.**

- Use MaterialApp/Scaffold around the screens.
- Open Details with Navigator.push<void> and MaterialPageRoute<void>.
- Keep Products in the stack and provide a pop/Back path.

Code response:

2. Login → Home with replacement/reset semantics.

- Login must not reappear through Back.
- Choose pushReplacement or pushAndRemoveUntil.
- State the resulting route history.

Code response:

3. Pass an immutable Course to a details constructor.

- Use required typed fields for title and summary.
- Do not pass BuildContext as the data model.
- Render the fields in the destination.

Code response:

4. Return a theme through a typed nullable Future.

- Caller awaits Navigator.push<String>.
- Destination returns Ocean/Violet/Amber with Navigator.pop(context, value).
- Handle null and mounted before showing feedback.

Code response:

5. Build a state-preserving NavigationBar shell.

- Use three destinations and store the selected index in State.
- Use IndexedStack for pages.
- Own and dispose a TextEditingController in State.

Code response:



6. Protect an unsaved edit screen with PopScope.

- Use a dirty flag and a duplicate-dialog guard.
- Ask Stay/Discard with AlertDialog.
- After confirmed discard, allow one controlled pop.

Code response:

7. Write the image selection flow.

- Use image_picker 1.2.3 and XFile?.
- Handle cancellation and mounted.
- Do not claim upload success until an upload response is received.

Code response:

8. Write a widget test for a typed navigation result.

- Open Details and choose Ocean.
- Call pumpAndSettle after asynchronous actions.
- Assert the final Home feedback.

Code response:



Q9. Draw and Trace Navigation State and Result Flow

1. Trace Products → Details → Back and annotate the route stack after each action.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

2. Draw the decision map from the required Back behavior to push, replacement, popUntil, or reset.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

3. Draw the PopScope state machine: clean/dirty → back attempt → dialog → Stay or Discard → optional pop.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

4. Trace typed constructor data versus RouteSettings.arguments and mark where validation is required.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

5. Trace a typed result: caller awaits → destination pops a value → caller handles null/mounted → feedback.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

6. Draw NavigationBar + IndexedStack and explain why the Search query survives a destination change.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:



7. Trace image selection: picker request → XFile or cancellation → lifecycle check → preview or unchanged state.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

8. Trace a widget test from Home to Details, result return, pumpAndSettle, and final assertion.

- Show the widget/route state, the action arrow, and the result or lifecycle decision.

Code response:

End of Lab 7 — Navigation in Flutter | Student Edition — No solutions included

